



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



第八章 图



- 图的基本概念
- 图的存储结构
- 图的遍历与连通性
- 最小生成树
- 最短路径
- 活动网络

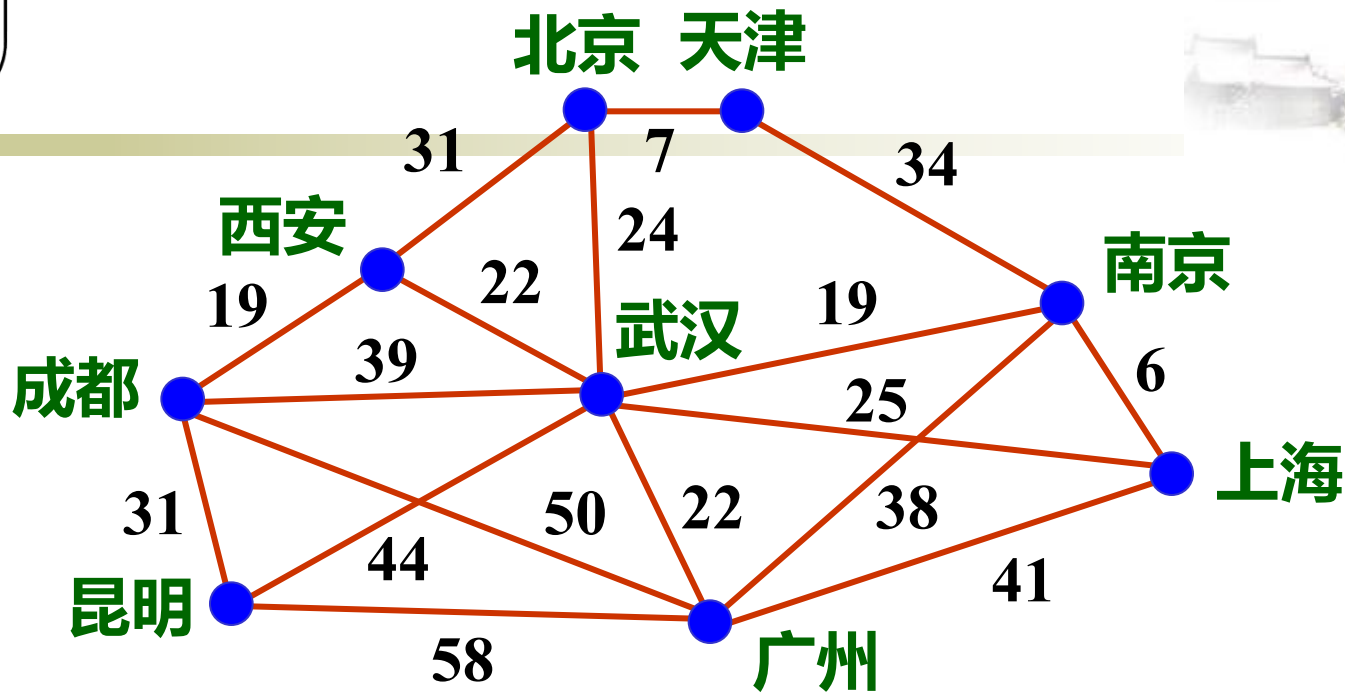




8.4 最小生成树 (minimum cost spanning tree)

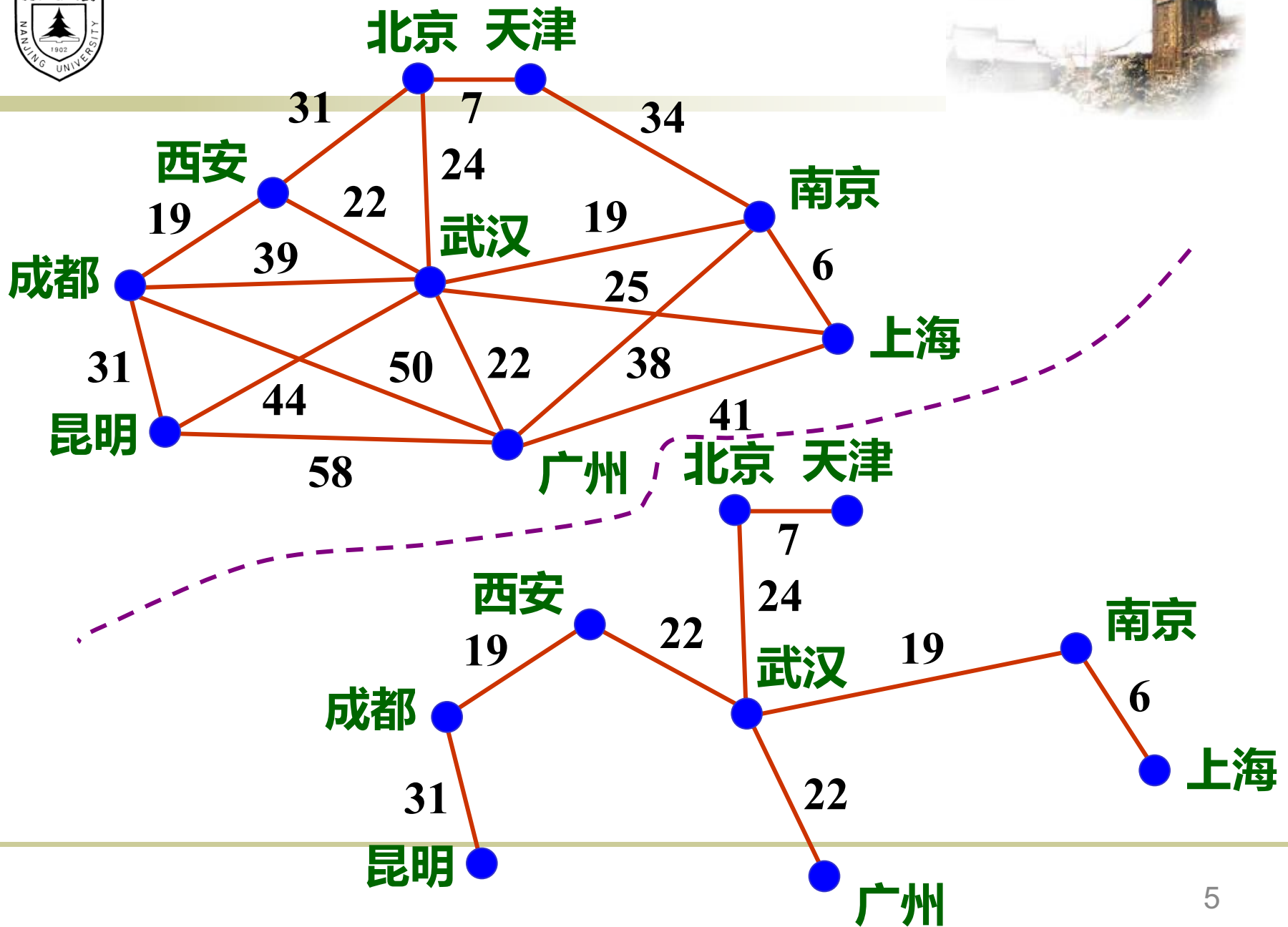


- 使用不同的遍历图的方法，可以得到不同的生成树；从不同的顶点出发，也可能得到不同的生成树。
- 按照生成树的定义， n 个顶点的连通网络的生成树有 n 个顶点、 $n-1$ 条边。
- **构造最小生成树**
假设有一个网络，用以表示 n 个城市之间架设通信线路，边上的权值代表架设通信线路的成本。
如何架设才能使线路架设的成本达到最小？



• 构造最小生成树的准则

- ❖ 必须使用且仅使用该网络中的 $n-1$ 条边来连接网络中的 n 个顶点;
- ❖ 不能使用产生回路的边;
- ❖ 各边上的权值的总和达到最小。





克鲁斯卡尔 (Kruskal) 算法



- 克鲁斯卡尔算法的基本思想：
设有一个有 n 个顶点的连通网络 $N = \{ V, E \}$, 最初先构造一个只有 n 个顶点、没有边的非连通图 $T = \{ V, \emptyset \}$, 图中每个顶点自成一个连通分量。
当在 E 中选到一条具有**最小权值**的边时, 若该边的两个顶点落在不同的连通分量上, 则将此边加入到 T 中; 否则将此边舍去, 重新选择一条权值最小的边。
如此重复下去, 直到所有顶点在同一个连通分量上为止。



Kruskal算法的伪代码描述



```
 $T = \emptyset;$  //  $T$ 是最小生成树的边集合  
//  $E$ 是带权无向图的边集合  
while (  $T$  包含的边少于  $n-1$  &&  $E$  不空) {  
    从  $E$  中选一条具有最小代价的边  $(v, w)$ ;  
    从  $E$  中删去  $(v, w)$ ;  
    如果  $(v, w)$  加到  $T$  中后不会产生回路, 则将  
         $(v, w)$  加入  $T$ ; 否则放弃  $(v, w)$ ;  
}  
if (  $T$  中包含的边少于  $n-1$  条)  
    cout << "不是最小生成树" << endl;
```



- 算法的框架 利用**最小堆**(MinHeap)和**并查集**(UFSets)来实现**克鲁斯卡尔算法**。
- 首先, 利用最小堆来存放 E 中所有的边, 堆中每个结点的格式为

tail	head	cost
------	------	------

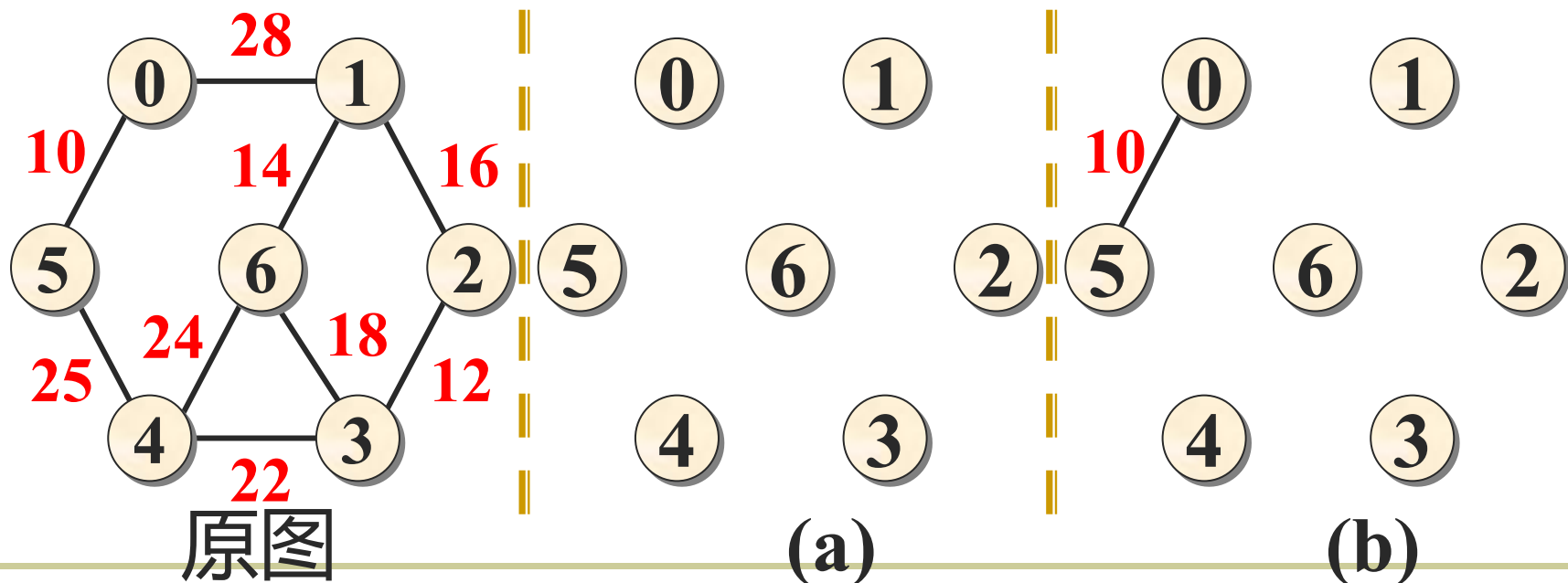
边的两个顶点位置 边的权值

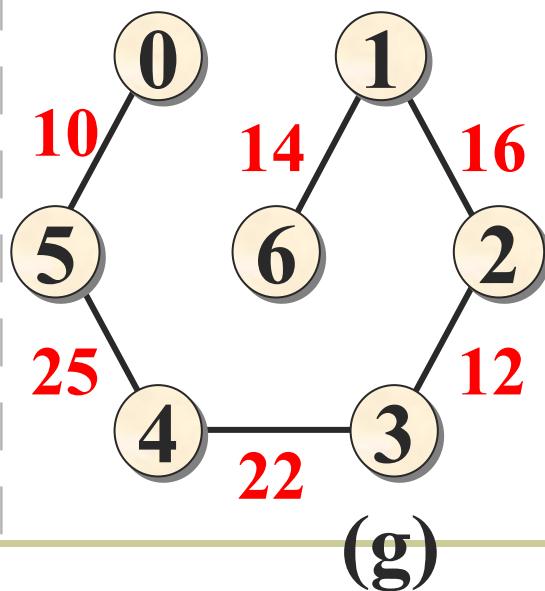
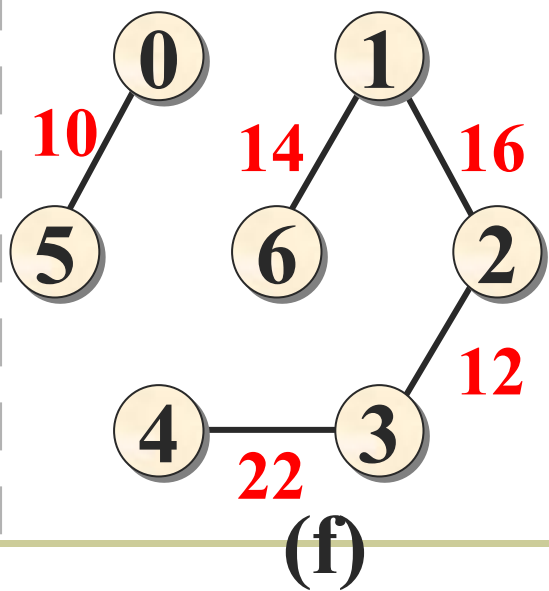
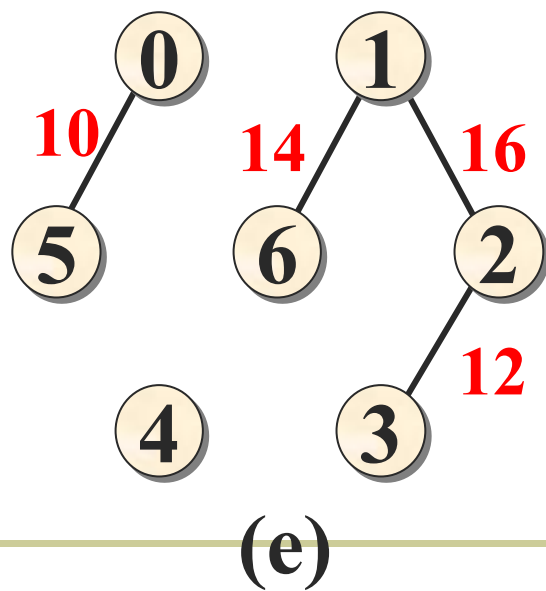
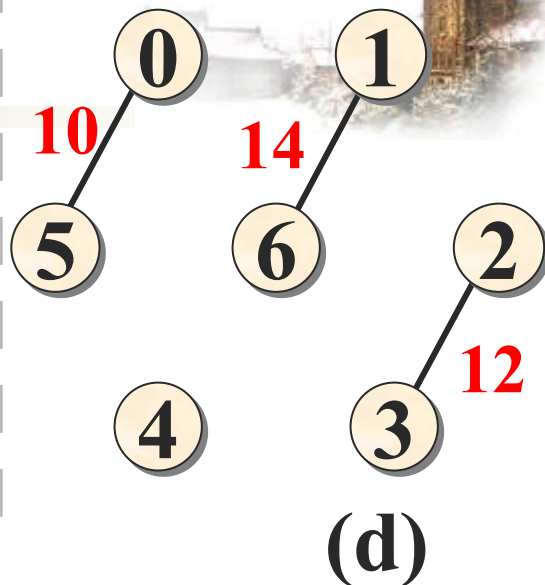
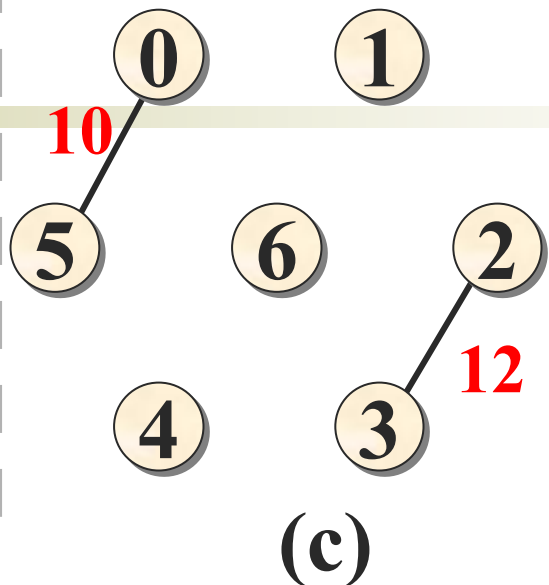
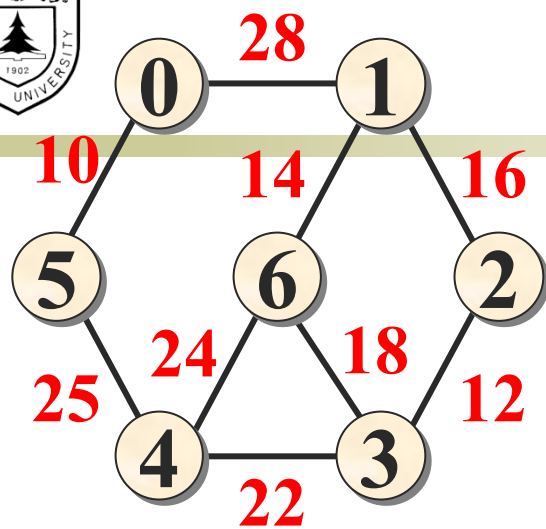
- 在构造最小生成树过程中, 利用并查集的运算检查依附一条边的两顶点tail、head 是否在同一连通分量 (即**并查集的同一个小集合**) 上, 是则舍去这条边; 否则将此边加入 T , 同时将这两个顶点放在同一个连通分量上。



- 随着各边逐步加入到最小生成树的边集合中，各连通分量也在逐步合并，直到形成一个连通分量为止。

构造最小生成树的过程







最小生成树类定义



```
const float maxValue = FLOAT_MAX
```

```
//机器可表示的、问题中不可能出现的大数
```

```
template <class T, class E>
```

```
struct MSTEdgeNode {
```

```
//树边结点的类定义
```

```
    int tail, head;
```

```
//两顶点位置
```

```
    E cost;
```

```
//边上的权值
```

```
    MSTEdgeNode() : tail(-1), head(-1), cost(0) { }
```

```
//构造函数
```

```
};
```



```
template <class T, class E>
class MinSpanTree { //树的类定义
protected:
    MSTEdgeNode<T, E> *edgevalue; //边值数组
    int maxSize;                  //最大元素个数
    int currentSize;              //当前元素个数
public:
    MinSpanTree (int sz = DefaultSize-1) : maxSize (sz),
        currentSize (0) {
        edgevalue = new MSTEdgeNode<T, E>[sz];
    }
    int Insert (MSTEdgeNode& item);
};
```



- 在求解最小生成树时，可以用邻接矩阵存储图，也可以用邻接表存储图。
- 算法中使用图的抽象基类的操作，无需考虑图及其操作的具体实现。

Kruskal算法的实现

```
#include "heap.h"
```

```
#include "UFSets.h"
```

```
template <class T, class E>
```

```
void Kruskal (Graph<T, E>& G,  
             MinSpanTree<T, E>& MST) {
```



```
MSTEdgeNode<T, E> edge;           //边结点辅助单元
int u, v, count;
int n = G.NumberOfVertices(); //顶点数
int m = G.NumberOfEdges();   //边数
MinHeap <MSTEdgeNode<T, E>> H(m); //最小堆
UFSets F(n);                 //并查集
for (u = 0; u < n; u++)
    for (v = u+1; v < n; v++)
        if (G.getWeight(u,v) != maxValue) { //插入堆
            edge.tail = u; edge.head = v;
            edge.cost = G.getWeight (u, v);
            H.Insert(edge);
        }
```



```
count = 1;           //最小生成树边数计数
while (count < n) {   //反复执行, 取n-1条边
    H.RemoveMin(edge); //退出具有最小权值的边
    u = F.Find(edge.tail); v = F.Find(edge.head);
                        //取两顶点所在集合的根u与v
    if (u != v) {
        //不是同一集合, 不连通
        F.Union(u, v);           //合并, 连通它们
        MST.Insert(edge);        //该边存入MST
        count++;
    }
}
}
```



出堆顺序

(1,6,14) 选中

(3,4,22) 选中

(0,5,10) 选中

(1,2,16) 选中

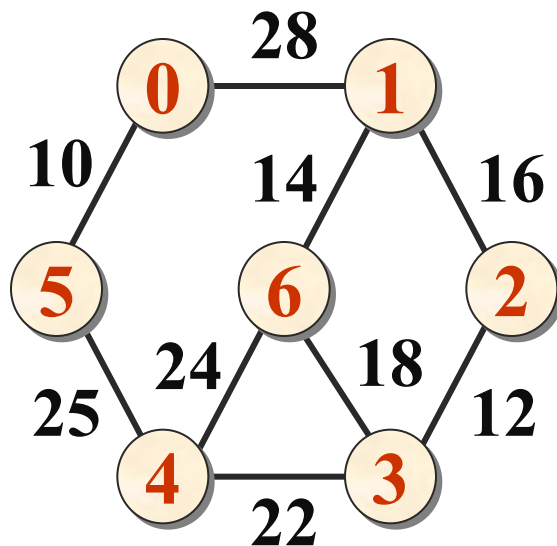
(4,6,24) 舍弃

(2,3,12) 选中

(3,6,18) 舍弃

(4,5,25) 选中

	0	1	2	3	4	5	6	
<i>F</i>	-1	-1	-1	-1	-1	-1	-1	
	-2	-1	-1	-1	-1	0	-1	(0,5,10)
	-2	-1	-2	2	-1	0	-1	(2,3,12)
	-2	-2	-2	2	-1	0	1	(1,6,14)
	-2	-4	1	2	-1	0	1	(1,2,16)
	-2	-5	1	2	1	0	1	(3,4,22)
	1	-7	1	2	1	0	1	(4,5,25)



原图

并查集



普里姆(Prim)算法



- 普里姆算法的基本思想:

从连通网络 $N = \{V, E\}$ 中的某一顶点 u_0 出发, 选择与它关联的具有**最小权值的边** (u_0, v) , 将其**顶点加入到生成树的顶点集合 U 中**。

以后**每一步**从一个顶点在集合 U 中而另一个顶点不在集合 U 中的各条边中**选择权值最小的边** (u, v) , 把它的顶点加入到**集合 U 中**。如此继续下去, 直到网络中的所有顶点都加入到生成树顶点集合 U 中为止。



普里姆(Prim)的伪代码描述



选定构造最小生成树的出发顶点 u_0 ;

$V_{mst} = \{u_0\}$, $E_{mst} = \emptyset$;

while (V_{mst} 包含的顶点少于 n && E 不空) {

 从 E 中选一条边 (u, v) ,

$u \in V_{mst} \cap v \in V - V_{mst}$, 且具有最小代价(cost);

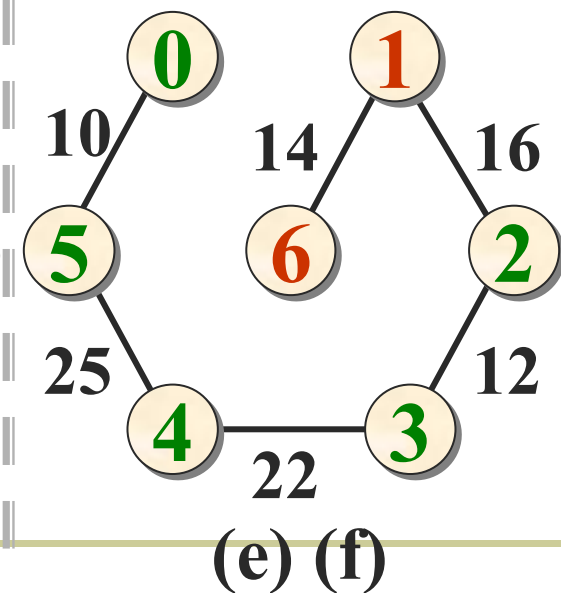
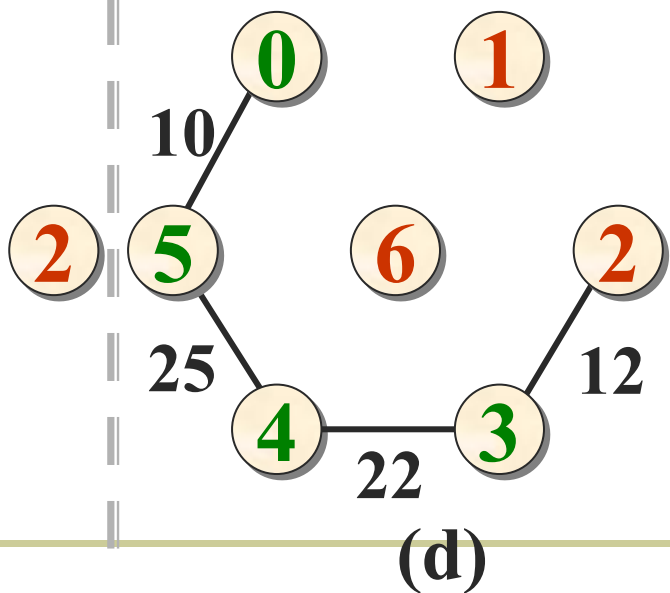
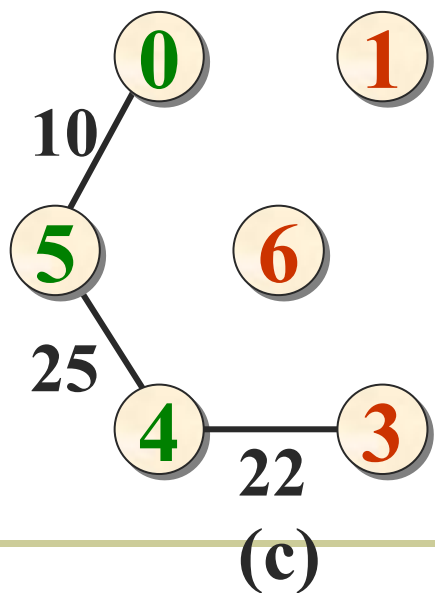
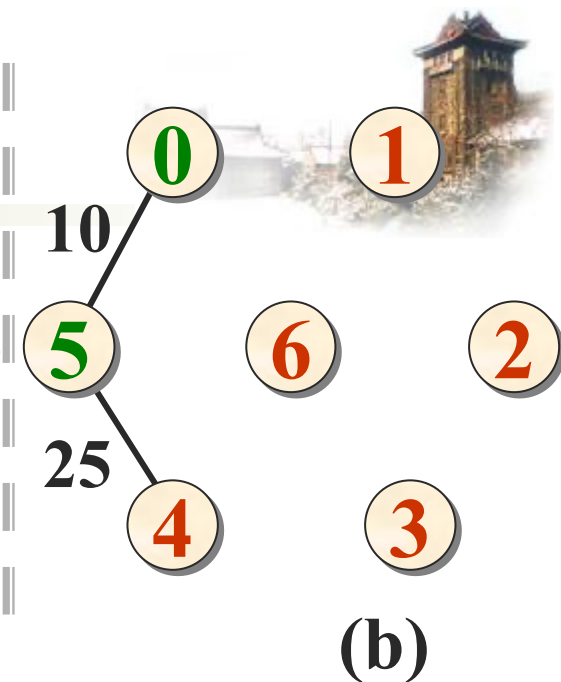
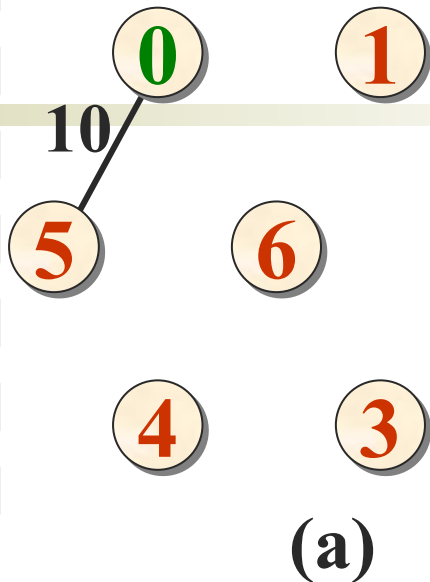
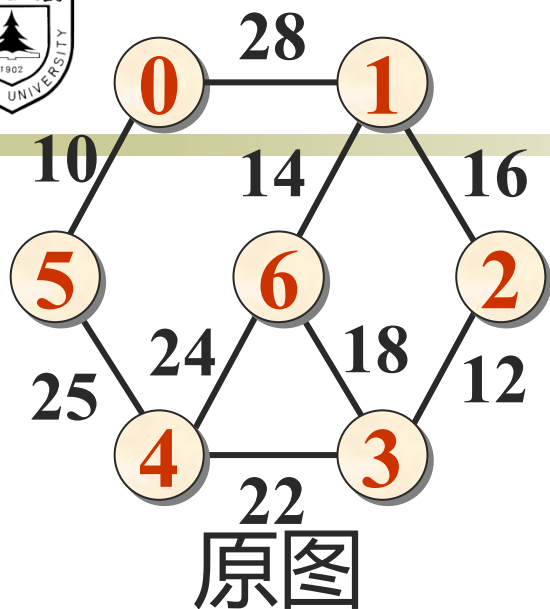
 令 $V_{mst} = V_{mst} \cup \{v\}$, $E_{mst} = E_{mst} \cup \{(u, v)\}$;

 将新选出的边从 E 中剔除: $E = E - \{(u, v)\}$;

}

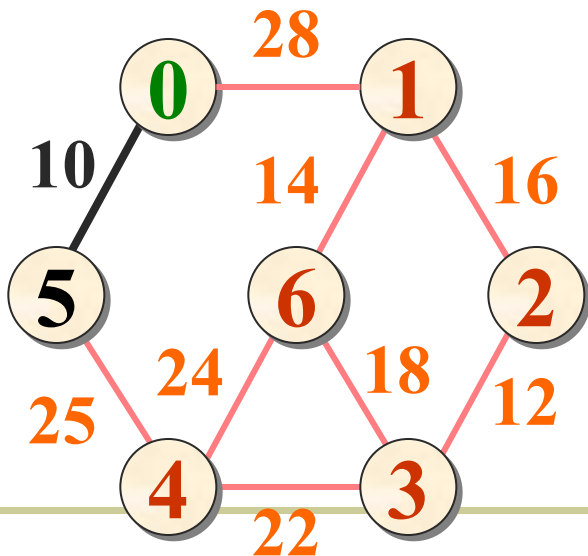
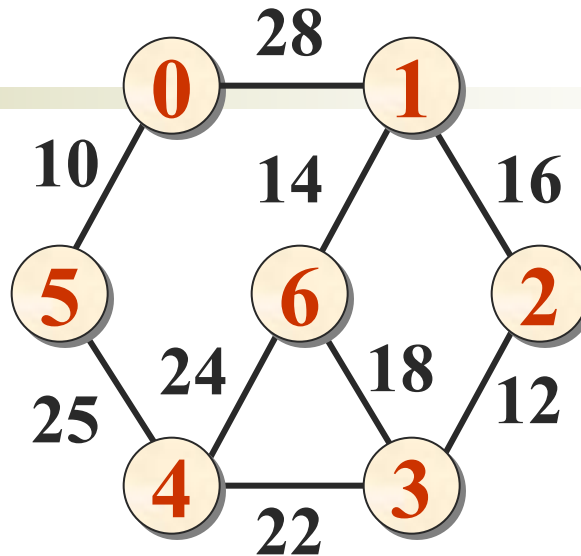
if (V_{mst} 包含的顶点少于 n)

 cout << "不是最小生成树" << endl;





例子



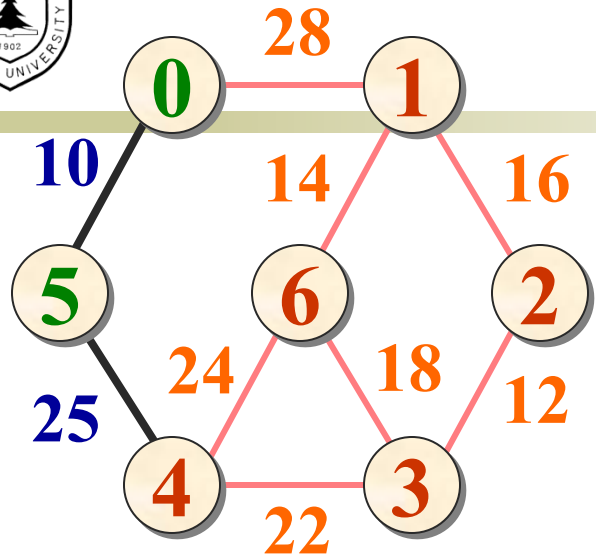
$$H = \{(0,5,10), (0,1,28)\}$$

$$\text{edge} = (0, 5, 10)$$

$$V_{\text{mst}} = \{t, f, f, f, f, f, f\}$$



$$V_{\text{mst}} = \{t, f, f, f, f, t, f\}$$



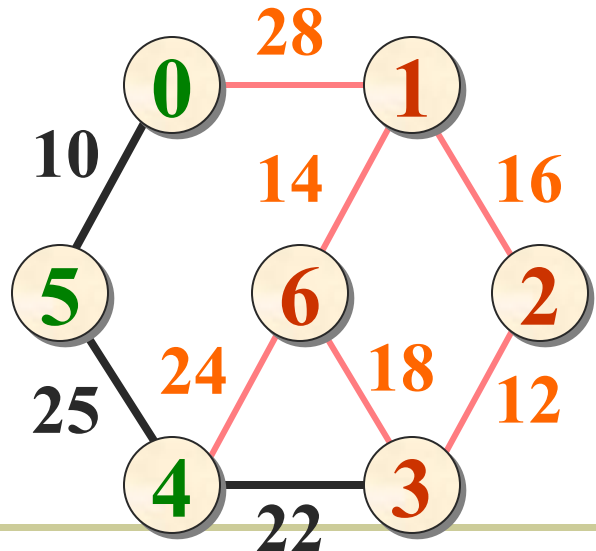
$$H = \{(5,4,25), (0,1,28)\}$$

$$\text{edge} = (5, 4, 25)$$

$$V_{\text{mst}} = \{t, f, f, f, f, t, f\}$$



$$V_{\text{mst}} = \{t, f, f, f, t, t, f\}$$



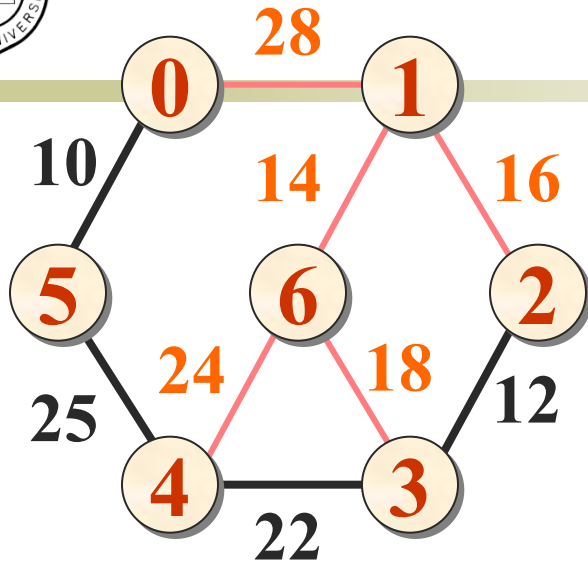
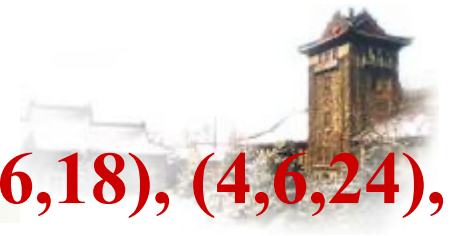
$$H = \{(4,3,22), (4,6,24), (0,1,28)\}$$

$$\text{edge} = (4, 3, 22)$$

$$V_{\text{mst}} = \{t, f, f, f, t, t, f\}$$



$$V_{\text{mst}} = \{t, f, f, t, t, t, f\}$$



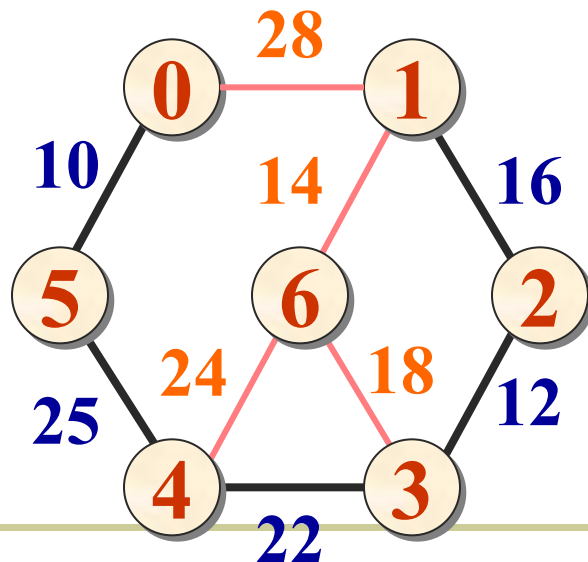
$H = \{(3,2,12), (3,6,18), (4,6,24), (0,1,28)\}$

edge = (3, 2, 12)

$V_{\text{mst}} = \{t, f, f, t, t, t, f\}$



$V_{\text{mst}} = \{t, f, t, t, t, t, f\}$



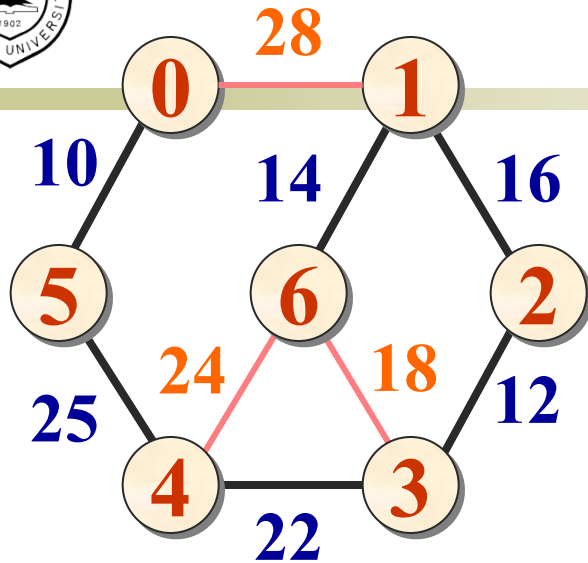
$H = \{(2,1,16), (3,6,18), (4,6,24), (0,1,28)\}$

edge = (2, 1, 16)

$V_{\text{mst}} = \{t, f, t, t, t, t, f\}$



$V_{\text{mst}} = \{t, t, t, t, t, t, f\}$



$H = \{(1,6,14), (3,6,18), (4,6,24), (0,1,28)\}$

$\text{edge} = (1, 6, 14)$

$V_{\text{mst}} = \{t, t, t, t, t, t, f\}$



$V_{\text{mst}} = \{t, t, t, t, t, t, t\}$

- 最小生成树中边集合里存入的各条边为：
(0, 5, 10), (5, 4, 25), (4, 3, 22),
(3, 2, 12), (2, 1, 16), (1, 6, 14)



Prim算法的实现



```
#include "heap.h"
template <class T, class E>
void Prim (Graph<T, E>& G, const T u0,
           MinSpanTree<T, E>& MST) {
    MSTEdgeNode<T, E> edge;    //边结点辅助单元
    int i, u, v, count;
    int n = G.NumberOfVertices();    //顶点数
    int m = G.NumberOfEdges();       //边数
    int u = G.getVertexPos(u0);      //起始顶点号
    MinHeap <MSTEdgeNode<T, E>> H(m); //最小堆
```




```
bool Vmst = new bool[n]; //最小生成树顶点集合
for (i = 0; i < n; i++) Vmst[i] = false;
Vmst[u] = true;           //u 加入生成树
count = 1;
do {                       //迭代
    v = G.getFirstNeighbor(u);
    while (v != -1) {      //检测u所有邻接顶点
        if (!Vmst[v]) {   //v不在mst中
            edge.tail = u; edge.head = v;
            edge.cost = G.getWeight(u, v);
            H.Insert(edge); // (u,v)加入堆
        } //堆中存所有u在mst中, v不在mst中的边
        v = G.getNextNeighbor(u, v);
    }
}
```



```
while (!H.IsEmpty() && count < n) {  
    H.RemoveMin(edge);    //选堆中具最小权的边  
    if (!Vmst[edge.head]) {  
        MST.Insert(edge);    //加入最小生成树  
        u = edge.head; Vmst[u] = true;  
                                //u加入生成树顶点集合  
  
        count++;  
        break;  
    }  
}  
};
```



- Kruskal算法是以边为对象，不断地加入新的不构成环路的最短边来构成最小生成树。
- Prim算法是以点为对象，挑选与点相连的最短边来构成最小生成树。
- Kruskal算法适合于边稀疏的情形。
- Prim算法适用于边稠密的网络。
- 注意：当各边有相同权值时，由于选择的随意性，产生的生成树可能不唯一。



对于一个带权连通无向图 $G = (V, E)$ ，生成树不同，每棵树的权（即树中所有边上的权值之和）也可能不同。设 R 为 G 的所有生成树的集合，若 T 为 R 中边的权值之和最小的生成树，则 T 称为 G 的最小生成树（*Minimum-Spanning-Tree, MST*）。

Prim 算法（普里姆）：

从某一个顶点开始构建生成树；
每次将代价最小的新顶点纳入生成树，直到所有顶点都纳入为止。

时间复杂度： $O(|V|^2)$
适合用于边稠密图

Kruskal 算法（克鲁斯卡尔）：

每次选择一条权值最小的边，使这条边的两头连通（原本已经连通的就不选）
直到所有结点都连通

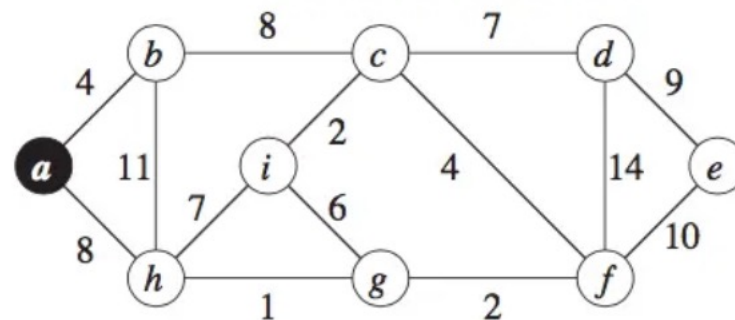
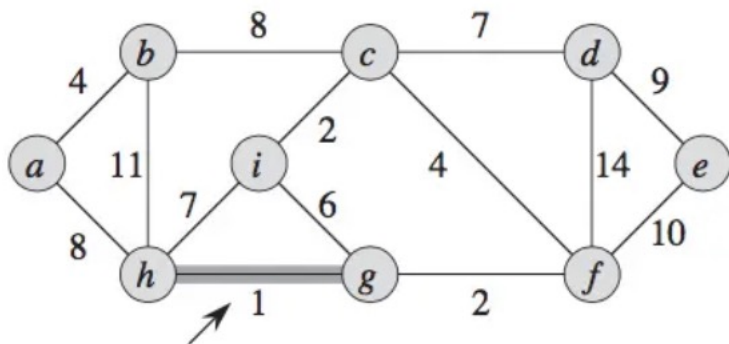
时间复杂度： $O(|E|\log_2|E|)$
适合用于边稀疏图



练习



求下图的最小生成树，分别采用Kruskal算法和Prim算法（已给出第一步）





最短路径 (Shortest Path)



- **最短路径问题：**如果从图中某一顶点（称为**源点**）到另一顶点（称为**终点**）的路径可能不止一条，如何找到一条路径使得沿此路径上各边上的权值总和达到最小。
- **问题解法**
 - ◆ **边上权值非负情形的单源最短路径问题**
 - Dijkstra算法（1972年图灵奖得主）
 - ◆ **边上权值为任意值的单源最短路径问题**
 - Bellman和Ford算法 ×（不讲）
 - ◆ **所有顶点之间的最短路径**
 - Floyd算法（1978年图灵奖得主）



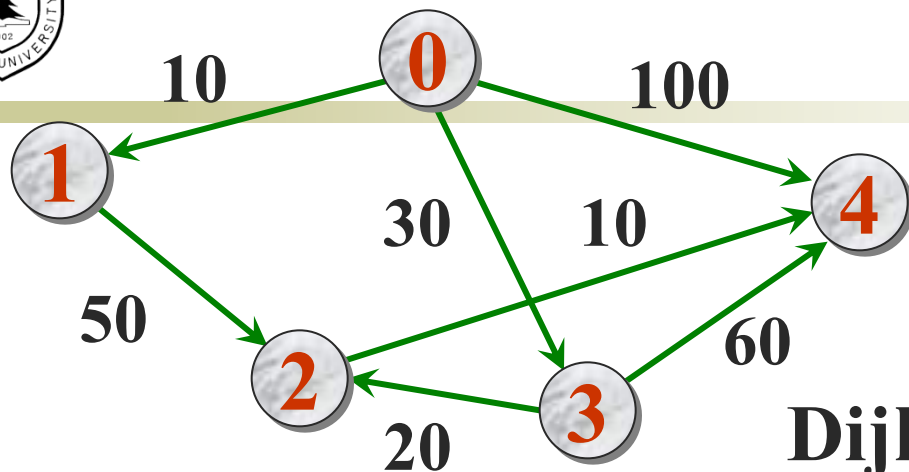
边上权值非负情形的 单源最短路径问题



- 问题的提法：给定一个带权有向图 D 与源点 v ，求从 v 到 D 中其他顶点的最短路径。限定各边上的权值大于或等于0。
- 应用：GPS导航
- 为求得这些最短路径，Dijkstra提出按路径长度的递增次序，逐步产生最短路径的算法。首先求出长度最短的一条最短路径，再参照它求出长度次短的一条最短路径，依次类推，直到从顶点 v 到其它各顶点的最短路径全部求出为止。



- Dijkstra算法用于求解带权有向图或无向图的单源最短路径问题，即从某一个源点出发，到图中所有其他顶点的最短路径。
- 前提条件： 所有权重必须非负
- 算法思想
 - 采用贪心策略，逐步扩展最短路径树：
 - 维护两个集合：已确定最短路径的顶点集合**S**和未确定最短路径的顶点集合**T**
 - 每次从**T**中选取距离源点最近的顶点加入**S**
 - 更新该顶点邻居的距离值



Dijkstra逐步求解的过程

源点	终点	最短路径	路径长度
v_0	v_1	(v_0, v_1)	10
	v_2	— (v_0, v_1, v_2) (v_0, v_3, v_2)	$\infty, 60, 50$
	v_3	(v_0, v_3)	30
	v_4	(v_0, v_3, v_4) (v_0, v_3, v_2, v_4)	100, 90, 60



- 引入**辅助数组dist**。它的每一个分量 $\text{dist}[i]$ 表示当前找到的从源点 v_0 到终点 v_i 的最短路径的长度。
初始状态：
 - ◆ 若从 v_0 到顶点 v_i 有边, 则 $\text{dist}[i]$ 为该边的权值;
 - ◆ 若从 v_0 到顶点 v_i 无边, 则 $\text{dist}[i]$ 为 ∞ 。
- 假设 S 是已求得的最短路径的终点的集合, 则可证明: 下一条最短路径必然是从 v_0 出发, 中间只经过 S 中的顶点便可到达的那些顶点 v_x ($v_x \in V - S$) 的路径中的一条。
- 每次求得一条最短路径后, 其终点 v_k 加入集合 S , 然后对所有的 $v_i \in V - S$, 修改其 $\text{dist}[i]$ 值。



Dijkstra算法可描述如下:

① 初始化: $S \leftarrow \{v_0\}$;

$\text{dist}[j] \leftarrow \text{Edge}[0][j]$, $j = 1, 2, \dots, n-1$;

// n 为图中顶点个数

② 求出最短路径的长度:

$\text{dist}[u] \leftarrow \min \{\text{dist}[i]\}$, $i \in V-S$;

$S \leftarrow S \cup \{u\}$;

③ 修改:

$\text{dist}[k] \leftarrow \min \{\text{dist}[k], \text{dist}[u] + \text{Edge}[u][k]\}$,

对于每一个 $k \in V-S$;

④ 判断: 若 $S = V$, 则算法结束, 否则转②。



Computation of Dijkstra on digraph followed

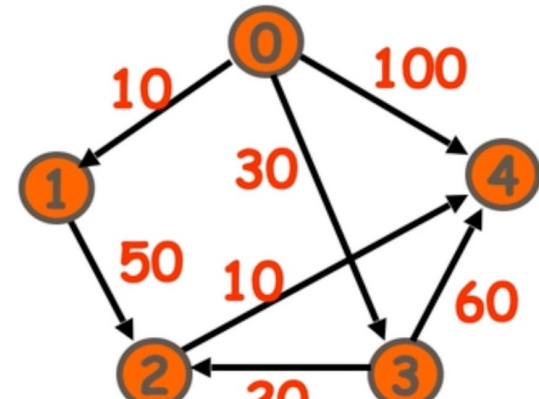
选取 终点	顶点1			顶点2			顶点3			顶点4		
	$S[1]$	$d[1]$	$p[1]$	$S[2]$	$d[2]$	$p[2]$	$S[3]$	$d[3]$	$p[3]$	$S[4]$	$d[4]$	$p[4]$
0	0	<u>10</u>	0	0	∞	0	0	30	0	0	100	0
1	1	10	0	0	60	1	0	<u>30</u>	0	0	100	0
3	1	10	0	0	<u>50</u>	3	1	30	0	0	90	3
2	1	10	0	1	50	3	1	30	0	0	<u>60</u>	2
4	1	10	0	1	50	3	1	30	0	1	60	2

how to reconstruct the shortest path
from the source to each vertex?

$path[4] = 2 \rightarrow path[2] = 3 \rightarrow$

$path[3] = 0$, Inverse it , 0, 3, 2, 4.

It is the shortest path from vertex 0 to 4.





计算从单个顶点到其他各顶点 最短路径的算法



```
void ShortestPath (Graph<T, E>& G, T v, E dist[], int path[])  
{  
    //Graph是一个带权有向图。dist[j],  $0 \leq j < n$ , 是当前  
    //求到的从顶点v到顶点j的最短路径长度, path[j],  
    //  $0 \leq j < n$ , 存放求到的最短路径。  
    int n = G.NumberOfVertices();  
    bool *S = new bool[n];    //最短路径顶点集  
    int i, j, k; E w, min;  
    for (i = 0; i < n; i++) {  
        dist[i] = G.getWeight(v, i);
```



```
S[i] = false;  
if (i != v && dist[i] < maxValue) path[i] = v;  
else path[i] = -1;  
}  
S[v] = true; dist[v] = 0;           //顶点v加入顶点集合  
for (i = 0; i < n-1; i++) {        //求解各顶点最短路径  
    min = maxValue; int u = v;  
    //选不在S中具有最短路径的顶点u  
    for (j = 0; j < n; j++)  
        if (!S[j] && dist[j] < min)  
            { u = j; min = dist[j];}  
    S[u] = true;                     //将顶点u加入集合S
```



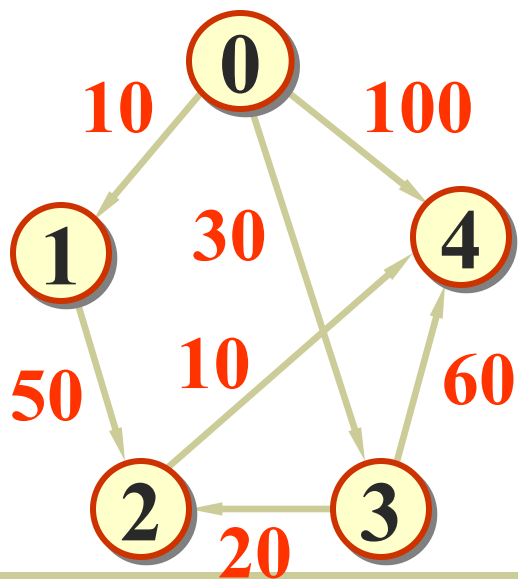
```
for (k = 0; k < n; k++) {           //修改
    w = G.GetWeight(u, k);
    if (!S[k] && w < maxValue &&
        dist[u]+w < dist[k]) {      //顶点k未加入S
        dist[k] = dist[u]+w;
        path[k] = u;                //修改到k的最短路径
    }
}
};
```



Dijkstra算法中各辅助数组的最终结果



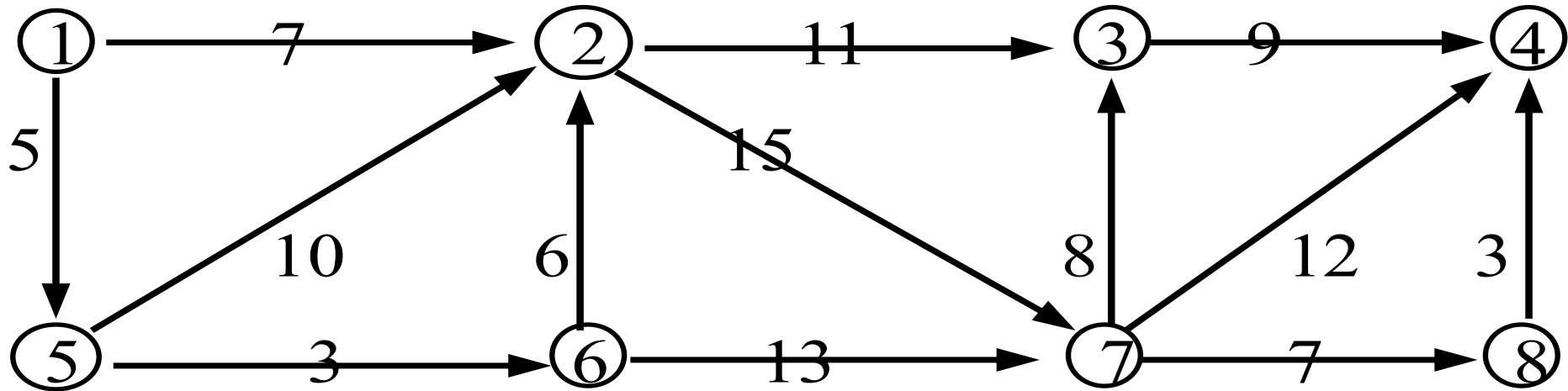
序号	顶点1	顶点2	顶点3	顶点4
Dist	10	50	30	60
path	0	3	0	2



从表中读取源点0到终点v的最短路径的方法：举顶点4为例
 $\text{path}[4] = 2 \rightarrow \text{path}[2] = 3 \rightarrow \text{path}[3] = 0$ ，反过来排列，得到路径 0, 3, 2, 4，这就是源点0到终点4的最短路径。

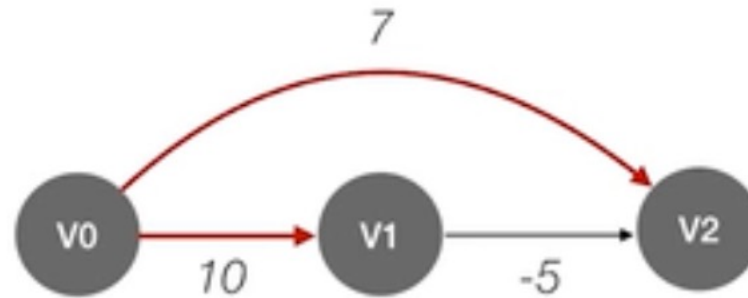


求：源点1到所有点的最短路径





不适用于带负权值的图





Floyd算法



- Floyd算法用于求解所有顶点对之间的最短路径，可以处理负权边，但不能处理负权环。
- 应用：城市距离矩阵
- 算法思想：动态规划

Floyd算法：求出每一对顶点之间的最短路径

使用动态规划思想，将问题的求解分为多个阶段

对于 n 个顶点的图 G ，求任意一对顶点 $V_i \rightarrow V_j$ 之间的最短路径可分为如下几个阶段：

#初始：不允许在其他顶点中转，最短路径是？

#0：若允许在 V_0 中转，最短路径是？

#1：若允许在 V_0 、 V_1 中转，最短路径是？

#2：若允许在 V_0 、 V_1 、 V_2 中转，最短路径是？

...

$n-1$ ：若允许在 V_0 、 V_1 、 V_2 V_{n-1} 中转，最短路径是？



所有顶点之间的最短路径



- 问题的提法: 已知一个各边权值均大于0的带权有向图, 对每一对顶点 $v_i \neq v_j$, 要求求出 v_i 与 v_j 之间的最短路径和最短路径长度。

- Floyd算法的基本思想:

定义一个 n 阶方阵序列:

$$A^{(-1)}, A^{(0)}, \dots, A^{(n-1)}.$$

其中 $A^{(-1)}[i][j] = \text{Edge}[i][j]$;

$$A^{(k)}[i][j] = \min \{ A^{(k-1)}[i][j],$$

$$A^{(k-1)}[i][k] + A^{(k-1)}[k][j] \}, k = 0, 1, \dots, n-1$$



- $A^{(0)}[i][j]$ 是从顶点 v_i 到 v_j , 中间顶点是 v_0 的最短路径长度;
- $A^{(k)}[i][j]$ 是从顶点 v_i 到 v_j , 中间顶点的序号不大于 k 的最短路径的长度;
- $A^{(n-1)}[i][j]$ 是从顶点 v_i 到 v_j 的最短路径长度。

求各对顶点间最短路径的算法

```
template <class T, class E>
```

```
void Floyd (Graph<T, E>& G, E a[][[]], int path[][[]]) {  
    //a[i][j]是顶点i和j之间的最短路径长度。 path[i][j]  
    //是相应路径上顶点j的前一顶点的顶点号。
```



```
for (i = 0; i < n; i++)           //矩阵a与path初始化
    for (j = 0; j < n; j++) {
        a[i][j] = G.getWeight (i, j);
        if (i != j && a[i][j] < max Value) path[i][j] = i;
        else path[i][j] = 0;
    }
for (k = 0; k < n; k++)
    //针对每一个k, 产生a(k)及path(k)
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            if (a[i][k] + a[k][j] < a[i][j]) {
                a[i][j] = a[i][k] + a[k][j];
```



```
path[i][j] = path[k][j];  
    //缩短路径长度, 绕过  $k$  到  $j$   
}
```

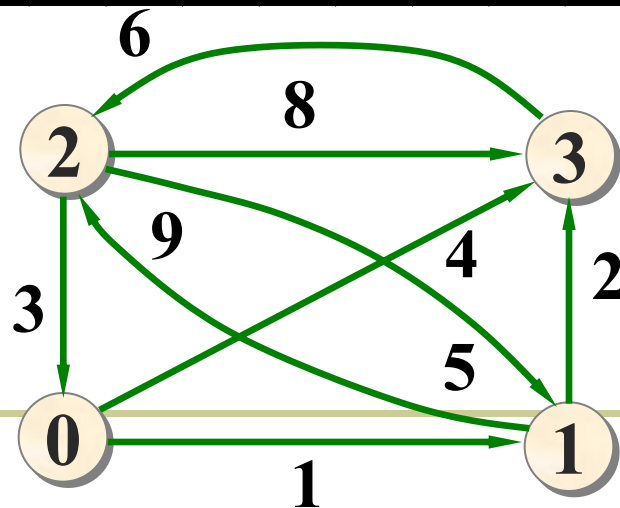
```
};
```

- **Floyd算法允许图中有带负权值的边，但不许有包含带负权值的边组成的回路。**



	$A^{(-1)}$				$A^{(0)}$				$A^{(1)}$				$A^{(2)}$				$A^{(3)}$			
	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3
0	0	1	∞	4	0	1	∞	4	0	1	10	3	0	1	10	3	0	1	9	3
1	∞	0	9	2	∞	0	9	2	∞	0	9	2	12	0	9	2	11	0	8	2
2	3	5	0	8	3	4	0	7	3	4	0	6	3	4	0	6	3	4	0	6
3	∞	∞	6	0	∞	∞	6	0	∞	∞	6	0	9	10	6	0	9	10	6	0

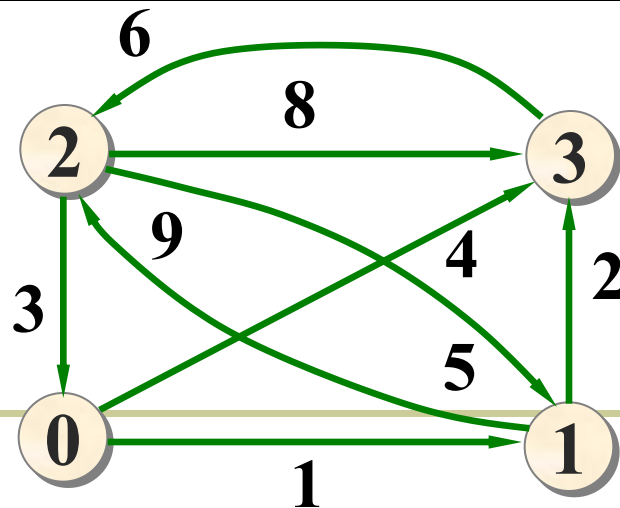
	$Path^{(-1)}$				$Path^{(0)}$				$Path^{(1)}$				$Path^{(2)}$				$Path^{(3)}$			
	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3
0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	1	1	0	0	3	1
1	0	0	1	1	0	0	1	1	0	0	1	1	2	0	1	1	2	0	3	1
2	2	2	0	2	2	0	0	0	2	0	0	1	2	0	0	1	2	0	0	1
3	0	0	3	0	0	0	3	0	0	0	3	0	2	0	3	0	2	0	3	0





	$A^{(-1)}$				$A^{(0)}$				$A^{(1)}$				$A^{(2)}$				$A^{(3)}$			
	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3
0	0	1	∞	4	0	1	∞	4	0	1	10	3	0	1	10	3	0	1	9	3
1	∞	0	9	2	∞	0	9	2	∞	0	9	2	12	0	9	2	11	0	8	2
2	3	5	0	8	3	4	0	7	3	4	0	6	3	4	0	6	3	4	0	6
3	∞	∞	6	0	∞	∞	6	0	∞	∞	6	0	9	10	6	0	9	10	6	0

	$Path^{(-1)}$				$Path^{(0)}$				$Path^{(1)}$				$Path^{(2)}$				$Path^{(3)}$			
	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3	0	1	2	3
0	-1	0	-1	0	-1	0	0	0	-1	0	1	1	-1	0	1	1	-1	0	3	1
1	-1	-1	1	1	-1	-1	1	1	-1	-1	1	1	2	-1	1	1	2	-1	3	1
2	2	2	-1	2	2	0	-1	0	2	0	-1	1	2	0	-1	1	2	0	-1	1
3	-1	-1	3	-1	-1	-1	3	-1	-1	-1	3	-1	2	0	3	-1	2	0	3	-1

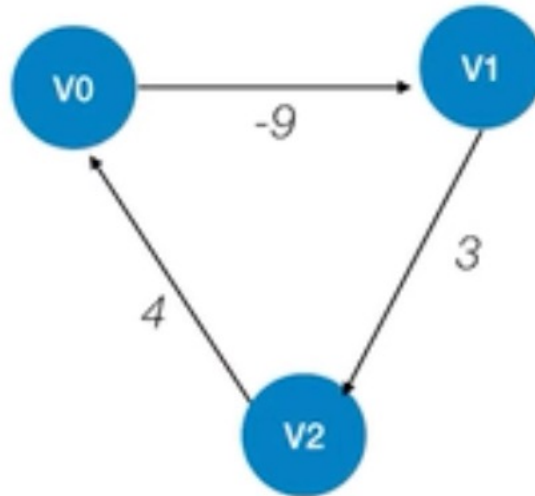




- 以 $\text{Path}^{(3)}$ 为例，对最短路径读法加以说明。从 $A^{(3)}$ 知，顶点1到0的最短路径长度为 $a[1][0] = 11$ ，其最短路径看 $\text{path}[1][0] = 2$ ， $\text{path}[1][2] = 3$ ， $\text{path}[1][3] = 1$ ，表示顶点0 $\leftarrow$ 顶点2 $\leftarrow$ 顶点3 $\leftarrow$ 顶点1；从顶点1到顶点0最短路径为 $\langle 1, 3 \rangle, \langle 3, 2 \rangle, \langle 2, 0 \rangle$ 。
- 本章给出的求解最短路径的算法不仅适用于带权有向图，对带权无向图也可以适用。因为带权无向图可以看作是有往返二重边的有向图。



Floyd不能解决的问题





	BFS 算法	Dijkstra 算法	Floyd 算法
无权图	✓	✓	✓
带权图	✗	✓	✓
带负权值的图	✗	✗	✓
<u>带负权回路的图</u>	✗	✗	✗
时间复杂度	$O(V ^2)$ 或 $O(V + E)$	$O(V ^2)$	$O(V ^3)$
通常用于	求无权图的单源最短路径	求带权图的单源最短路径	求带权图中各顶点间的最短路径

注：也可用 Dijkstra 算法求所有顶点间的最短路径，重复 $|V|$ 次即可，总的时间复杂度也是 $O(|V|^3)$



■ 关于 **Dijkstra** 算法，以下说法正确的是：

- A) 它可以处理带有负权边的图，并且保证找到最短路径
- B) 它采用动态规划的思想，计算所有顶点对之间的最短路径
- C) 它采用贪心策略，逐步确定从源点到其他顶点的最短路径
- D) 它的时间复杂度为 $O(V^3)$ ，适合处理大规模稠密图

■ **Floyd** 算法的时间复杂度是 $O(V^3)$ ，主要原因是：

- A) 需要为每个顶点维护一个优先队列
- B) 需要进行 V 轮松弛操作，每轮操作需要遍历所有顶点对
- C) 需要多次执行深度优先遍历来检测负权环
- D) 需要对比多种不同的贪心策略

■ 在以下哪种场景下，应该选择使用 **Floyd** 算法而不是 **Dijkstra** 算法？

- A) 在一个包含 10,000 个顶点的社交网络中，寻找某个用户到所有其他用户的最短关系链
- B) 在一个包含 100 个城市的交通网络中，需要预计算所有城市之间的最短距离
- C) 在一个地图导航系统中，实时计算从当前位置到目的地的最短路径
- D) 在一个所有权重都为非负的大规模网络中进行单源最短路径查询